

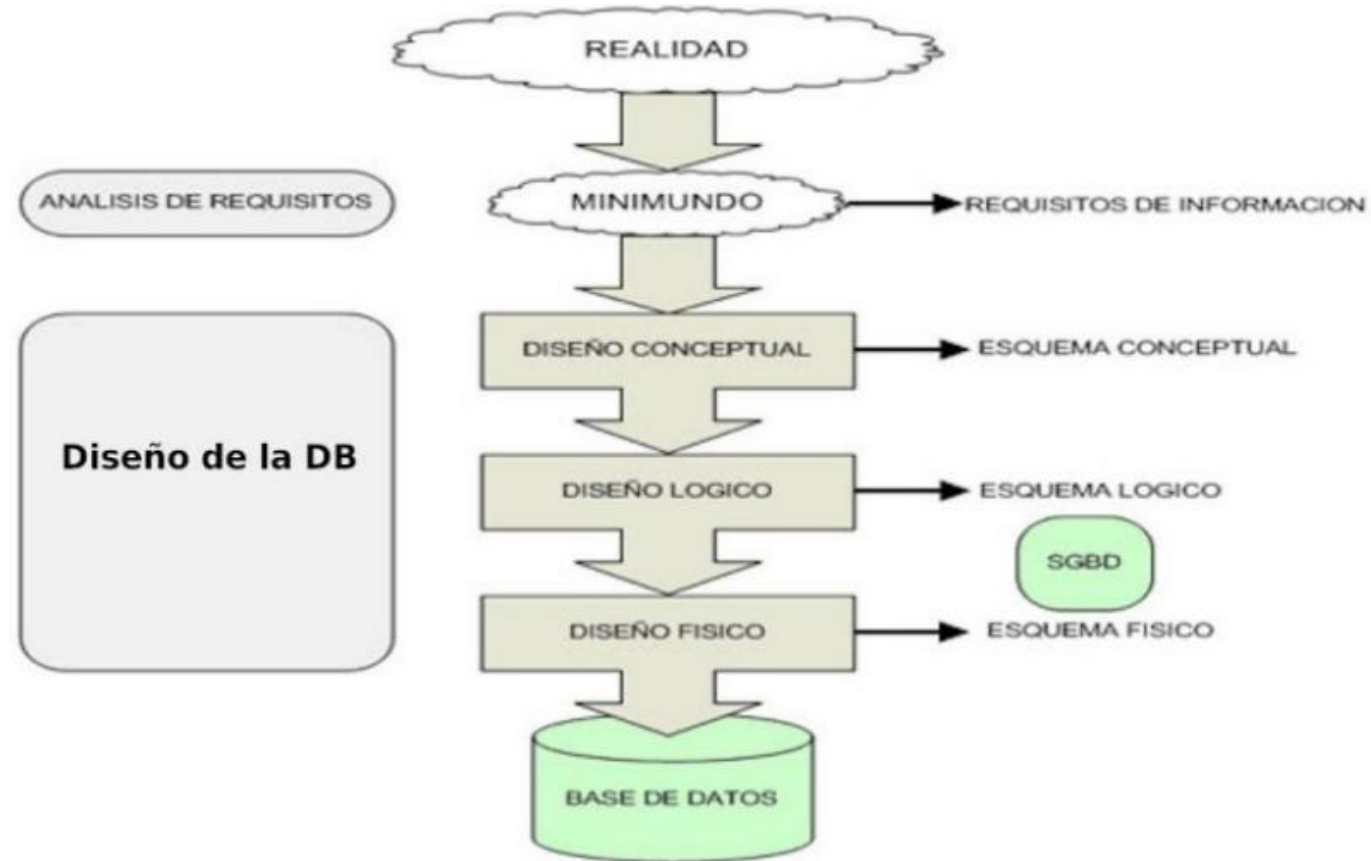


UT3: DISEÑO FÍSICO Y LÓGICO DE BD

Néstor Sabater Hernández

DISEÑO CONCEPTUAL

1. INTRODUCCIÓN



1. INTRODUCCIÓN

- Fases del Diseño de BD
 1. Recolección y análisis de requerimientos
 - Los diseñadores entrevistan a los futuros usuarios de la BD.
 - Se recoge y documenta sus necesidades de información.
 - Se definen requerimientos funcionales:
 - Operaciones (transacciones) que se aplicarán a la base de datos.
 - Obtención de datos y actualización por parte de los usuarios.
-

1. INTRODUCCIÓN

- Fases del Diseño de BD

2. Diseño Conceptual

- Crear un esquema conceptual
- Modelo entidad-relación
 - Contiene descripción detallada de:
 - Los requerimientos de información de los usuarios
 - Los tipos de datos
 - Relaciones entre ellos
 - Restricciones.

1. INTRODUCCIÓN

- Fases del Diseño de BD

3. Diseño Lógico

- Implementar la BD con un SGBD comercial
- Transformar el modelo conceptual al modelo de datos que use el SGBD (jerárquico, red o relacional)

1. INTRODUCCIÓN

- Fases del Diseño de BD

- 4. Diseño Físico

- Objetivo
 - Describir cómo se va a implementar físicamente el esquema lógico obtenido en la fase anterior.
 - En el modelo relacional, esto consiste en:
 - Obtener tablas, relaciones y restricciones
 - Determinar las estructuras de almacenamiento - Tipos de fichero y organización
 - Determinar los métodos de acceso
 - Diseñar el modelo de seguridad del sistema

1. INTRODUCCIÓN

- Fases del Diseño de BD

- 4. Diseño Físico

- SGBD son sistemas que permiten la creación, gestión y administración de bases de datos
 - Los más populares son:
 - MySQL
 - Microsoft SQL Server
 - Oracle
 - Microsoft Access
 - PostgreSQL
 - DB₂



1. INTRODUCCIÓN



- PostgreSQL
 - Las **características** principales de PostgreSQL son:
 - Es una base de datos relacional.
 - Es Open Source.
 - Es muy rápida, segura y fácil de usar.
 - Es una de las más usadas en Internet.
 - Existe una gran cantidad de software que la usa.

2. INSTALACIÓN

- Requisitos
 - Vamos a trabajar con **Windows**
 - Instalar PostgreSQL
 - <https://www.postgresql.org/>
 - Lanzar Stack Builder
 - Habilitar los Database Drivers
 - Usar pgAdmin4
-

3. HERRAMIENTAS

- Intérprete de comandos de los SGBD
 - **La utilidad principal de un SGBD es su intérprete de comandos.**
 - Es una aplicación cliente cuya única misión es enviar comandos al SGBD y mostrar los resultados devueltos por el SGBD en pantalla.
 - Trabajaremos con pgAdmin 4 (Query Tool). Todas las interacciones serán mediante sentencias SQL.
 - Para invocarlos desde el sistema operativo (Windows o Unix), tan solo hay que escribir en un terminal su nombre con ciertas opciones
-

4. SENTENCIAS SQL BÁSICAS

- COMENZAMOS

- Hay que tener en cuenta que en PostgreSQL **TODAS** las sentencias finalizan en ;

4. SENTENCIAS SQL BÁSICAS

- `select current_database();`
 - Comprobamos que BD está en uso
- `select datname from pg_database;`
 - Muestra las bases de datos que hay activas.

4. SENTENCIAS SQL BÁSICAS

- `create database nombreDeBD;`
 - Esta sentencia te permitirá crear una BD con el nombre que establezcas en `nombreDeBD`
 - `drop database nombreDeBD;`
 - Eliminará la BD con el nombre que se establezca en `nombreDeBD`
 - `select current_user;`
 - Mirar el usuario que está en uso
 - `select version();`
 - Mirar la versión
-

4. SENTENCIAS SQL BÁSICAS

- `select now();`
 - Muestra día y fecha actual del sistema
 - `select version(), now();`
 - Muestra en dos columnas la información que hemos puesto separada por una coma.
 - `select version(); select now();`
 - Se ejecutan como dos sentencias diferentes, ya que hay un ; que separa ambas sentencias.
-

Function Description	Function Description
<code>current_catalog</code> → name <code>current_database()</code> → name Returns the name of the current database. (Databases are called “catalogs” in the SQL standard, so <code>current_catalog</code> is the standard's spelling.)	<code>inet_client_port()</code> → integer Returns the IP port number of the current client, or NULL if the current connection is via a Unix-domain socket.
<code>current_query()</code> → text Returns the text of the currently executing query, as submitted by the client (which might contain more than one statement).	<code>inet_server_addr()</code> → inet Returns the IP address on which the server accepted the current connection, or NULL if the current connection is via a Unix-domain socket.
<code>current_role</code> → name This is equivalent to <code>current_user</code> .	<code>inet_server_port()</code> → integer Returns the IP port number on which the server accepted the current connection, or NULL if the current connection is via a Unix-domain socket.
<code>current_schema</code> → name <code>current_schema()</code> → name Returns the name of the schema that is first in the search path (or a null value if the search path is empty). This is the schema that will be used for any tables or other named objects that are created without specifying a target schema.	<code>pg_backend_pid()</code> → integer Returns the process ID of the server process attached to the current session.
<code>current_schemas(include_implicit boolean)</code> → name[] Returns an array of the names of all schemas presently in the effective search path, in their priority order. (Items in the <code>current_search_path</code> setting that do not correspond to existing, searchable schemas are omitted.) If the Boolean argument is <code>true</code> , then implicitly-searched system schemas such as <code>pg_catalog</code> are included in the result.	<code>pg_blocking_pids(integer)</code> → integer[] Returns an array of the process ID(s) of the sessions that are blocking the server process with the specified process ID from acquiring a lock, or an empty array if there is no such server process or it is not blocked. One server process blocks another if it either holds a lock that conflicts with the blocked process's lock request (hard block), or is waiting for a lock that would conflict with the blocked process's lock request and is ahead of it in the wait queue (soft block). When using parallel queries the result always lists client-visible process IDs (that is, <code>pg_backend_pid</code> results) even if the actual lock is held or awaited by a child worker process. As a result of that, there may be duplicated PIDs in the result. Also note that when a prepared transaction holds a conflicting lock, it will be represented by a zero process ID. Frequent calls to this function could have some impact on database performance, because it needs exclusive access to the lock manager's shared state for a short time.
<code>current_user</code> → name Returns the user name of the current execution context.	<code>pg_conf_load_time()</code> → timestamp with time zone Returns the time when the server configuration files were last loaded. If the current session was alive at the time, this will be the time when the session itself re-read the configuration files (so the reading will vary a little in different sessions). Otherwise it is the time when the postmaster process re-read the configuration files.
<code>inet_client_addr()</code> → inet Returns the IP address of the current client, or NULL if the current connection is via a Unix-domain socket.	<code>pg_current_logfile([text])</code> → text Returns the path name of the log file currently in use by the logging collector. The path includes the <code>log_directory</code> directory and the individual log file name. The result is NULL if the logging collector is disabled. When multiple log files exist, each in a different format, <code>pg_current_logfile</code> without an argument returns the path of the file having the first format found in the ordered list: <code>stderr</code> , <code>csvlog</code> , <code>jsonlog</code> . NULL is returned if no log file has any of these formats. To request information about a specific log file format, supply either <code>csvlog</code> , <code>jsonlog</code> or <code>stderr</code> as the value of the optional parameter. The result is NULL if the log format requested is not configured in <code>log_destination</code> . The result reflects the contents of the <code>current_logfiles</code> file. This function is restricted to superusers and roles with privileges of the <code>pg_monitor</code> role by default, but other users can be granted EXECUTE to run the function.
	<code>pg_get_loaded_modules()</code> → setof record(<i>module_name</i> text, <i>version</i> text, <i>file_name</i> text) Returns a list of the loadable modules that are loaded into the current server session. The <i>module_name</i> and <i>version</i> fields are NULL unless the module author supplied values for them using the <code>PG_MODULE_MAGIC_EXT</code> macro. The <i>file_name</i> field gives the file name of the module (shared library).

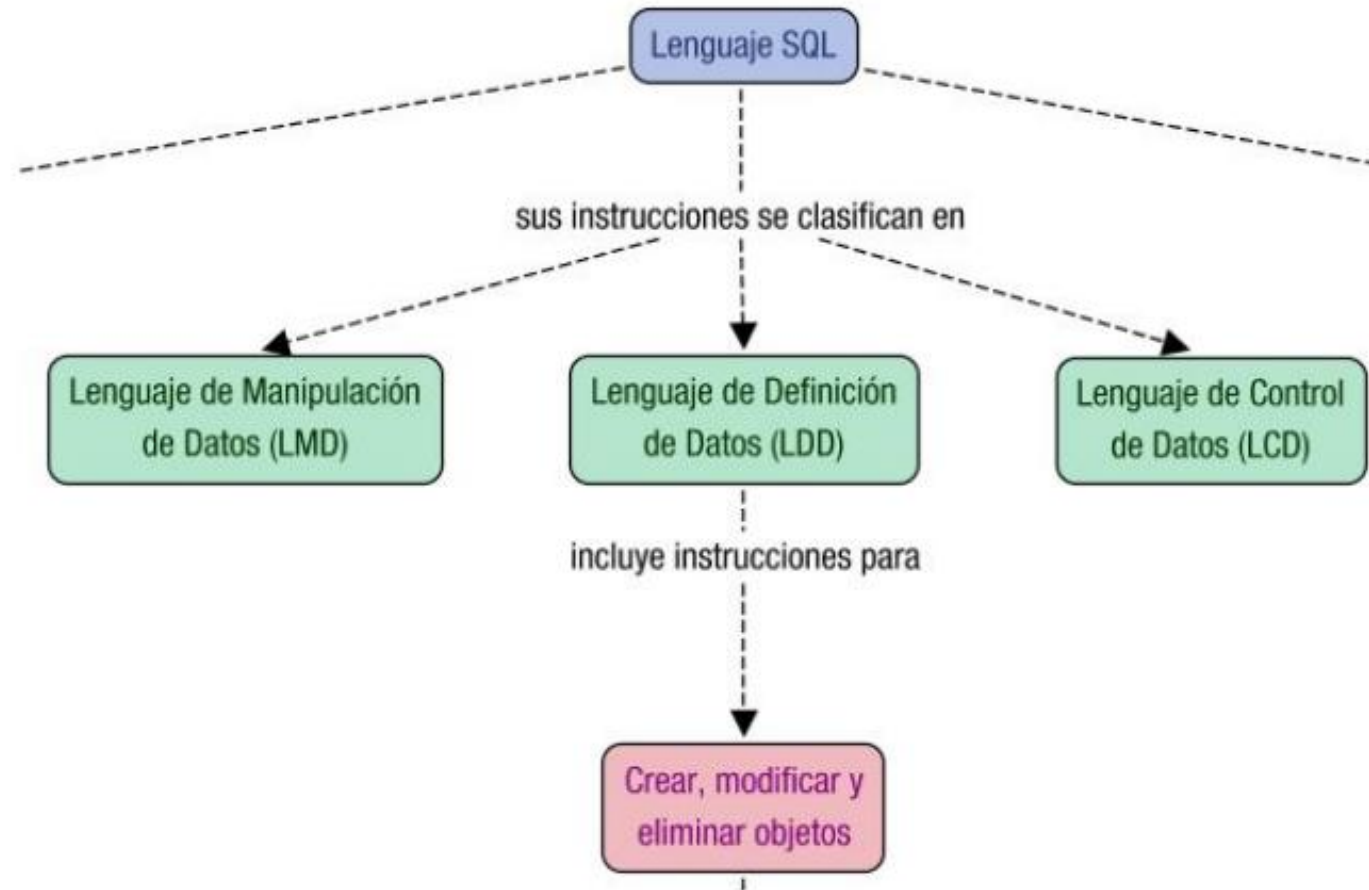
4. SENTENCIAS SQL BÁSICAS

Function	Description
<code>pg_my_temp_schema () → oid</code>	Returns the OID of the current session's temporary schema, or zero if it has none (because it has not created any temporary tables).
<code>pg_is_other_temp_schema (oid) → boolean</code>	Returns true if the given OID is the OID of another session's temporary schema. (This can be useful, for example, to exclude other sessions' temporary tables from a catalog display.)
<code>pg_jit_available () → boolean</code>	Returns true if a JIT compiler extension is available (see Chapter 30) and the jit configuration parameter is set to on.
<code>pg_numa_available () → boolean</code>	Returns true if the server has been compiled with NUMA support.
<code>pg_listening_channels () → setof text</code>	Returns the set of names of asynchronous notification channels that the current session is listening to.
<code>pg_notification_queue_usage () → double precision</code>	Returns the fraction (0–1) of the asynchronous notification queue's maximum size that is currently occupied by notifications that are waiting to be processed. See LISTEN and NOTIFY for more information.
<code>pg_postmaster_start_time () → timestamp with time zone</code>	Returns the time when the server started.
<code>pg_safe_snapshot_blocking_pids (integer) → integer[]</code>	Returns an array of the process ID(s) of the sessions that are blocking the server process with the specified process ID from acquiring a safe snapshot, or an empty array if there is no such server process or it is not blocked. A session running a <code>SERIALIZABLE</code> transaction blocks a <code>SERIALIZABLE READ ONLY DEFERRABLE</code> transaction from acquiring a snapshot until the latter determines that it is safe to avoid taking any predicate locks. See Section 13.2.3 for more information about serializable and deferrable transactions. Frequent calls to this function could have some impact on database performance, because it needs access to the predicate lock manager's shared state for a short time.
<code>pg_trigger_depth () → integer</code>	Returns the current nesting level of PostgreSQL triggers (0 if not called, directly or indirectly, from inside a trigger).
<code>session_user → name</code>	Returns the session user's name.
<code>system_user → text</code>	Returns the authentication method and the identity (if any) that the user presented during the authentication cycle before they were assigned a database role. It is represented as <code>auth_method:identity</code> or NULL if the user has not been authenticated (for example if Trust authentication has been used).
<code>user → name</code>	This is equivalent to <code>current_user</code> .

5. LENGUAJE DE DEFINICIÓN DE DATOS

- **Lenguajes de Base de Datos**
 - Lenguaje proporcionado por el SGBD
 - Objetivo: Comunicarse con la BD
 - Estándar: SQL

5. LENGUAJE DE DEFINICIÓN DE DATOS



5. LENGUAJE DE DEFINICIÓN DE DATOS

- **Lenguajes de Base de Datos**

- Tipos

- **Lenguajes de Definición de Datos (LDD)**

- Orientados a la creación de la BD.
 - Modificaciones y cambios en su estructura física.

- **Lenguajes de Manipulación de Datos (LMD)**

- Gestión de la BD
 - Consultas, inserciones, eliminaciones y modificaciones de datos

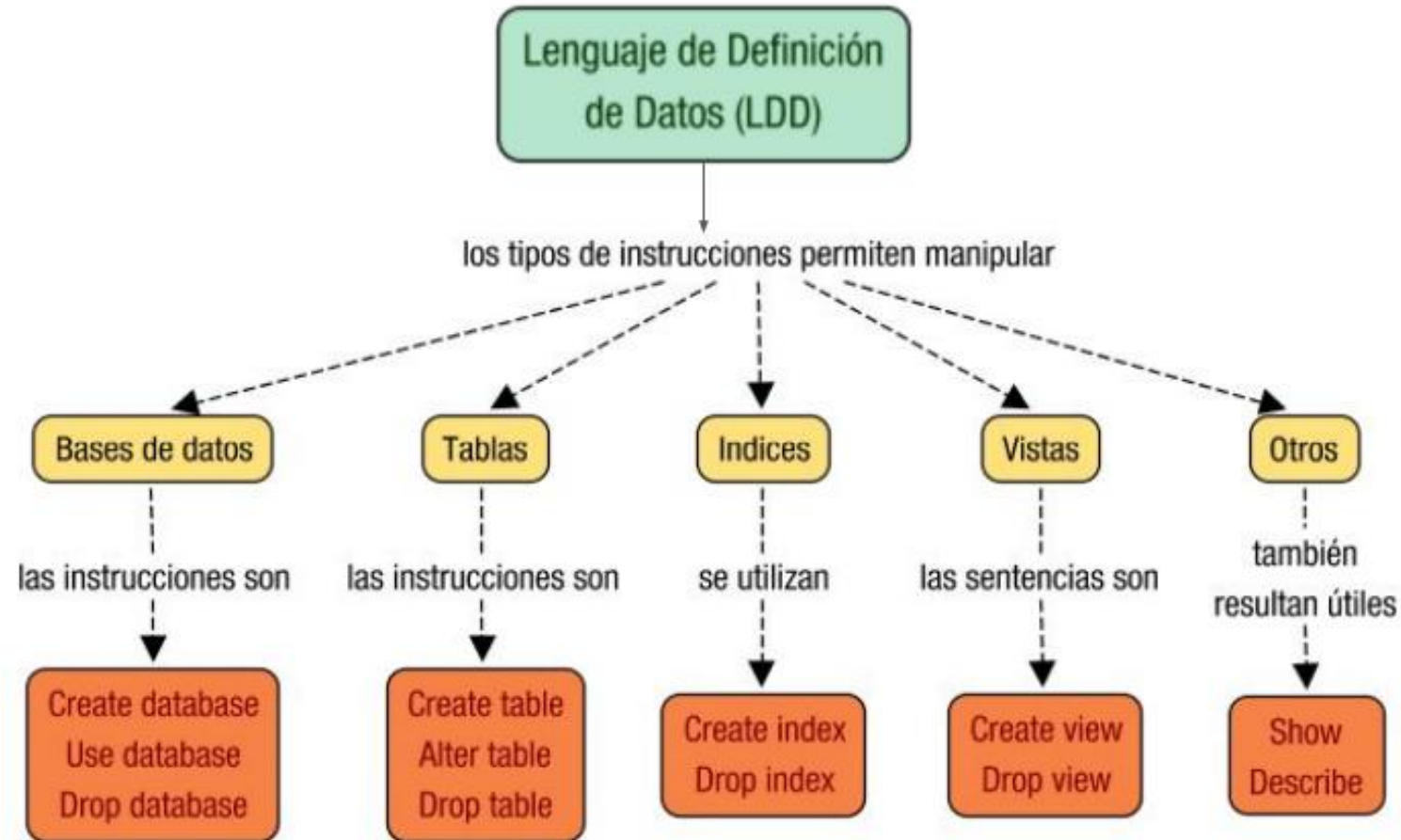
- **Lenguaje de control de datos (LCD)**

- Control de privilegios de usuarios
-

5. LENGUAJE DE DEFINICIÓN DE DATOS

- El sublenguaje de SQL que permite la definición de datos es el **DDL** (Data Definition Language).
 - Las funciones de este sublenguaje son:
 - Crear tablas, índices y otros objetos de la base de datos (como vistas, sinónimos, etc.)
 - Definir las estructuras físicas donde se almacenarán los objetos de las bases de datos (espacios de tablas (tablespaces), ficheros de datos (datafiles), etc.)
-

5. LENGUAJE DE DEFINICIÓN DE DATOS



5. LENGUAJE DE DEFINICIÓN DE DATOS

- INSTRUCCIONES

- **CREATE** **tipo_objeto** **Nombre** **Definición**

- Crea un **objeto** de un determinado tipo
 - DATABASE, TABLE, INDEX, etc.
 - con un **nombre** (por ejemplo Actores o Personajes)
 - una **definición, atributos...**(CodigoPersonaje, Nombre, etc.).

5. LENGUAJE DE DEFINICIÓN DE DATOS

- INSTRUCCIONES

- **DROP tipo_objeto Nombre**

- Elimina un **tipo de objeto** especificado mediante un nombre.
 - Por ejemplo, la sentencia:
 - DROP TABLE Actores
 - Borraría de la base de datos la tabla Actores junto con todos sus datos.

5. LENGUAJE DE DEFINICIÓN DE DATOS

- INSTRUCCIONES
 - **ALTER tipo_objeto Nombre {Modificación}**
 - Modifica la definición de un objeto.
 - Por ejemplo, la sentencia
 - ALTER TABLE Actores DROP COLUMN Fecha
 - Eliminaría la columna Fecha de la tabla Actores

5. LENGUAJE DE DEFINICIÓN DE DATOS

- NOTA SINTAXIS

- Para facilitar qué parte de la sintaxis se debe respetar y cuál se puede variar:
 - Las palabras reservadas del lenguaje se verán escritas en mayúsculas (CREATE, DATE, VARCHAR)
 - Y los nombres de objetos (Actores, Grado, Nombre) en minúsculas.

- EJEMPLO:

- CREATE TABLE Actores (Codigo INTEGER PRIMARY KEY, Nombre VARCHAR(40), Fecha DATE);
-

6. DDL DE BASES DE DATOS

- Creación de Bases de Datos
 - `CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] nombre;`
 - `CREATE DATABASE` crea una base de datos con el nombre dado.
 - Para usar `CREATE DATABASE` se necesita el privilegio `CREATE` en la base de datos.
 - Se producirá un error si la base de datos ya existe y no se ha especificado `IF NOT EXISTS`.
-

6. DDL DE BASES DE DATOS

- Borrado de Bases de Datos

```
1 DROP {DATABASE | SCHEMA} [IF EXISTS] db_name
```

- DROP DATABASE Proveedores;

7. DDL DE TABLAS

- Crear Tablas

```
mysql> create table personas (  
-> nombre char(30),  
-> dirección char(40),  
-> teléfono char(15)  
-> );
```

- CUIDADO! FALTAN RESTRICCIONES
-

7. DDL DE TABLAS

- Crear Tablas

- Sintaxis reducida

```
1 CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
2     (create_definition, ...)
3     [table_options]
4     [partition_options]
```

- (Create_definition)

- NombreColumna TipoDeDatos Restricciones

- Opciones de Tabla

7. DDL DE TABLAS

```
1 CREATE TABLE personas(  
2     nombre varchar(40) NOT NULL,  
3     direccion varchar(50) NULL,  
4     edo_civil char(13) DEFAULT 'Soltero',  
5     num_registro integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY  
6 );
```

- Restricciones de columna (Atributos)

Atributo	Significado
null	Se permiten valores nulos, atributos por omisión si no se especifica lo contrario.
not null	No se permiten valores nulos.
default valor	Valor por omisión que se asigna a la columna.
integer generated always as identity integer generated by default as identity serial	El valor se asigna automáticamente incrementando en uno el máximo valor registrado hasta ahora. Se aplica sólo a las columnas marcadas como clave primaria.
primary key	Señala al campo como clave primaria, implícitamente también lo declara como not null .

7. DDL DE TABLAS

- Restricciones de tabla

```
1 CREATE TABLE personas(  
2 nombre varchar(40) NOT NULL,  
3 nacimiento date NOT NULL,  
4 pareja varchar(40),  
5 proveedor int NOT NULL,  
6  
7 PRIMARY KEY (nombre, nacimiento),  
8 UNIQUE (pareja),  
9 FOREIGN KEY (proveedor) REFERENCES proveedores (codigo)  
10 );|
```

Restricción	Significado
primary key	Define la o las columnas que servirán como clave primaria. Las columnas que forman parte de la clave primaria deben ser not null .
unique	Define las columnas en las que no pueden duplicarse valores. Serán claves candidatas del modelo relacional.
foreign key (columna) references tabla (columna2)	Define que los valores de la <i>columna</i> se permitirán sólo si existen en <i>tabla</i> (<i>columna2</i>). Es decir, <i>columna</i> hace referencia a los registros de <i>tabla</i> , esto asegura que no se realicen referencias a registros que no existen.

7. DDL DE TABLAS

- FOREIGN KEY - REFERENCES

```
1  [CONSTRAINT [symbol]] FOREIGN KEY
2      [index_name] (col_name, ...)
3      REFERENCES tbl_name (col_name, ...)
4      [ON DELETE reference_option]
5      [ON UPDATE reference_option]
6
7  reference_option:
8      RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT
```

7. DDL DE TABLAS

- FOREIGN KEY – Acciones en FK
 - FOREIGN KEY (id) REFERENCES tabla(tabla_id) [ON DELETE opción] | [ON UPDATE opción]
 - Las opciones ON DELETE y ON UPDATE expresan las opciones de borrado y de eliminación sobre las filas de la tabla principal relacionada, es decir, qué ocurre en la tabla que estamos creando si se intenta modificar o eliminar filas en la tabla principal que están relacionadas con estas.
-

7. DDL DE TABLAS

- FOREIGN KEY – Referencias
 - FOREIGN KEY: Este tipo de restricción indica que las columnas indicadas entre paréntesis forman una clave ajena relativa a las columnas de otra tabla que se indica tras la palabra REFERENCES.
 - [CONSTRAINT [símbolo]]
 - **FOREIGN KEY** (ColumnaClaveForánea) **REFERENCES** NombreTabla(NombreColumna) [ON DELETE opción] | [ON UPDATE opción]
 - NombreTabla es la tabla a la que hace referencia la clave ajena
 - Las opciones ON DELETE y ON UPDATE expresan las opciones de borrado y de eliminación sobre las filas de la tabla principal relacionada
 - Es decir, qué ocurre en la tabla que estamos creando si se intenta modificar o eliminar filas en la tabla principal que están relacionadas con estas.
-

7. DDL DE TABLAS

- FOREIGN KEY – Referencias



7. DDL DE TABLAS

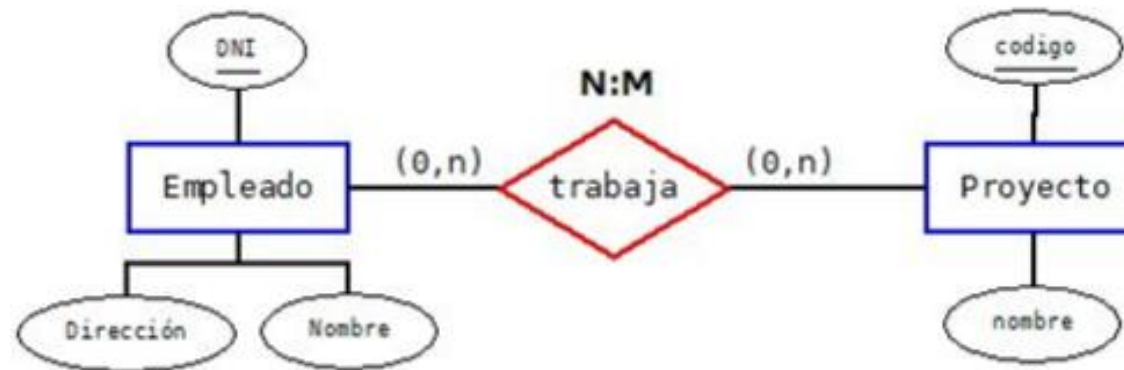
- FOREIGN KEY – Referencias y acciones en FK
 - **RESTRICT**
 - Es el comportamiento por defecto
 - Impide realizar modificaciones que atentan contra la integridad referencial.
 - Significa que no se puede realizar la acción si hay filas relacionadas en la tabla secundaria.
 - **CASCADE:**
 - Borra los registros de la tabla dependiente cuando se borra el registro de la tabla principal (en una sentencia DELETE)
 - Actualiza el valor de la clave secundaria cuando se actualiza el valor de la clave referenciada (en una sentencia UPDATE).
-

7. DDL DE TABLAS

- FOREIGN KEY – Referencias y acciones en FK
 - **SET NULL:**
 - Establece a NULL el valor de la clave secundaria cuando se elimina el registro en la tabla principal o se modifica el valor del campo referenciado.
 - **NO ACTION:**
 - En PostgreSQL es parecido a RESTRICT.
 - Permite las operaciones temporalmente, pero está mas pensado para transacciones.
 - Si al final de la transacción no se ha resuelto el problema que pueda haber surgido en la integridad, falla.
-

EJEMPLO

- No se puede eliminar o actualizar el DNI de ningún empleado



- EMPLEADOS (DNI, DIRECCIÓN, NOMBRE)
 - PROYECTOS (CÓDIGO, NOMBRE)
 - TRABAJOS (DNIempleado*, CODproyecto*)
-

EJEMPLO

- No se puede eliminar o actualizar el DNI de ningún empleado
 - EMPLEADOS (DNI, DIRECCIÓN, NOMBRE)
 - PROYECTOS (CÓDIGO, NOMBRE)
 - TRABAJOS (DNIempleado*, CODproyecto*)
 - CREATE TABLE EMPLEADOS(
 - DNI CHAR(9) PRIMARY KEY,
 - DIRECCION VARCHAR(30),
 - NOMBRE VARCHAR(25));
-

EJEMPLO

- No se puede eliminar o actualizar el DNI de ningún empleado
 - EMPLEADOS (DNI, DIRECCIÓN, NOMBRE)
 - PROYECTOS (CÓDIGO, NOMBRE)
 - TRABAJOS (DNIempleado*, CODproyecto*)
 - CREATE TABLE PROYECTOS(
 - CODIGO INT PRIMARY KEY,
 - NOMBRE VARCHAR(25));
-

EJEMPLO

- No se puede eliminar o actualizar el DNI de ningún empleado
 - EMPLEADOS (DNI, DIRECCIÓN, NOMBRE)
 - PROYECTOS (CÓDIGO, NOMBRE)
 - TRABAJOS (DNIempleado*, CODproyecto*)
 - CREATE TABLE TRABAJOS(
 - DNI_EMPLEADO CHAR(9) NOT NULL,
 - COD_PROYECTO INT NOT NULL,
 - PRIMARY KEY (DNI_EMPLEADO, COD_PROYECTO),
 - FOREIGN KEY (DNI_EMPLEADO) REFERENCES EMPLEADOS(DNI),
 - FOREIGN KEY (COD_PROYECTO) REFERENCES PROYECTOS(CODIGO));
-

7. DDL DE TABLAS

- Crear tablas temporales

- Sintaxis reducida

```
1 CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
2   (create_definition, ...)
3   [table_options]
4   [partition_options]
```

- Se puede utilizar la palabra clave TEMPORAL al crear una tabla.
 - Una tabla TEMPORAL es visible sólo dentro de la sesión actual y se elimina automáticamente cuando se cierra la sesión.

- Ejemplo: `mysql> create temporary table nueva_tabla ...`

- Este tipo de tablas pueden servir para realizar pruebas o copias de tablas existentes en nuestra BD.
-

7. DDL DE TABLAS

- Modificar tablas
 - La sentencia alter table permite una amplia gama de formas de modificar una tabla.

```
ALTER TABLE personal  
ADD COLUMN mascota char(20) DEFAULT 'perro',  
ADD COLUMN pasatiempo char(20) NOT NULL;
```

- Después de ejecutar la sentencia anterior, aparecen dos nuevas columnas en la tabla.
-

7. DDL DE TABLAS

- Modificar tablas
 - Si queremos agregar una sola columna, podemos usar la sintaxis siguiente:

```
ALTER TABLE personal ADD COLUMN capital int NOT NULL;
```

- Este formato de alter table en PostgreSQL no permite insertar las columnas antes o después de una columna a elección del usuario.
 - Si se desea reorganizar las columnas de una tabla, hay que volverla a crear y volcar la información de nuevo.
-

7. DDL DE TABLAS

- Modificar tablas

- Las columnas no deseadas pueden eliminarse con la opción drop.

```
ALTER TABLE personal DROP COLUMN pasatiempo;
```

- Esta versión se utilizará para borrar también las dependencias. (FK, vistas, etc.)

```
ALTER TABLE personal DROP COLUMN pasatiempo CASCADE;
```

- Cambiar de nombre la tabla:

```
ALTER TABLE personal RENAME TO gente;
```

7. DDL DE TABLAS

- Modificar columnas

- Cambiar nombre de una columna.

```
ALTER TABLE personal RENAME COLUMN nom TO nombre;
```

- Cambiar tipo de dato de una columna.

```
ALTER TABLE personal  
ALTER COLUMN edad TYPE int;
```

```
ALTER TABLE tabla  
ALTER COLUMN columna SET NOT NULL;
```

```
ALTER TABLE tabla  
ALTER COLUMN columna DROP NOT NULL;
```

- Hay otro tipo de modificaciones a tener en cuenta...

```
ALTER TABLE tabla  
ALTER COLUMN columna SET DEFAULT valor;
```

```
ALTER TABLE tabla  
ALTER COLUMN columna DROP DEFAULT;
```

7. DDL DE TABLAS

- Copiar tablas
 - Es posible crear una tabla nueva con el contenido de otra ya existente (copiando los datos):

```
mysql> create table nueva_tabla select * from otra_tabla;
```

- La siguiente sentencia es equivalente, pero no copia los datos de la tabla origen:

```
mysql> create table nueva_tabla like otra_tabla;
```

7. DDL DE TABLAS

- REPASO

Agregar columnas a tablas:	Agregar una sola columna:
ALTER TABLE personal ADD COLUMN mascota char(30) DEFAULT `perro`, ADD COLUMN pasatiempo char(20) NOT NULL;	ALTER TABLE personal ADD COLUMN capital int NOT NULL;
Eliminar columnas:	Editar columnas (nombre):
ALTER TABLE personal DROP COLUMN pasatiempo; ALTER TABLE personal DROP COLUMN pasatiempo CASCADE;	ALTER TABLE personal RENAME COLUMN nom TO nombre;
Cambiar nombre de tabla:	Editar columnas (tipo):
ALTER TABLE personal RENAME TO gente;	ALTER TABLE personal ALTER COLUMN edad TYPE int;

ACTIVIDAD 1

- Creación y manipulación de tablas

8. TIPOS DE DATOS

- PostgreSQL cuenta con un rico conjunto de tipos de datos para las columnas, que es necesario conocer para elegir mejor cómo definir las tablas.
 - Los tipos de datos se pueden clasificar en tres grupos:
 - Numéricos.
 - Cadenas de caracteres.
 - Booleanos.
 - Fechas y horas.
 - ENUM.
-

8. TIPOS DE DATOS

- Numéricos
 - Atributos específicos para columnas de tipo numérico:

Tipo	Uso	Tamaño
SMALLINT	Enteros pequeños	2 bytes
INTEGER (INT)	Enteros estándar	4 bytes
BIGINT	Enteros grandes	8 bytes
NUMERIC (p,s)	Decimales exactos	Variable
REAL, DOUBLE PRECISION	Decimales aproximados	4, 8 bytes

8. TIPOS DE DATOS

- Alfanuméricos
 - CHAR
 - Único tipo de dato para almacenar cadenas de longitud fija
 - Mayor rapidez de acceso a los datos
 - Si se intenta guardar en ellas valores que sobrepasen esos límites, lanza un error.
 - Cuando los valores CHAR se almacenan, se añaden espacios a la derecha hasta la longitud específica.
 - VARCHAR
 - Longitud variable
 - Menor tamaño de la base de datos
 - TEXT
 - Cadenas de texto plano longitud variable de longitud ilimitada.
 - Se comporta igual que VARCHAR.

8. TIPOS DE DATOS

- Booleanos
 - BOOLEAN
 - El tipo booleano puede tener varios estados: 'verdadero', 'falso' y un tercer estado, 'desconocido', que se representa con el valor nulo (NULL) de SQL.

8. TIPOS DE DATOS

- Fecha/Hora
 - DATE
 - Indica la fecha.
 - TIME
 - Indica la hora.
 - TIMESTAMP
 - Indica la fecha y hora.
 - TIMESTAMP WITH TIME ZONE
 - Indica la fecha y hora con la zona.

8. TIPOS DE DATOS

- ENUM
 - Tipo de cadena
 - Los valores se deben elegir de una lista (conjunto fijo de cadenas)
 - Ejemplo:
 - `CREATE TYPE estado_pedido AS ENUM ('pendiente', 'enviado', 'cancelado');`
 - `CREATE TABLE pedidos (`
 - `estado estado_pedido`
 - `);`
 - La columna `edo_civil` solo podrá almacenar los valores 'soltero', 'casado', 'viudo', 'divorciado'
 - La columna ocupará el espacio de un byte, ya que los valores enum son representados internamente por números.
-

8. TIPOS DE DATOS

- ENUM
 - Tipo de cadena
 - Los valores se deben elegir de una lista (conjunto fijo de cadenas)
 - Es un tipo independiente
 - Se reutiliza
-

8. TIPOS DE DATOS

- ENUM

```
CREATE TYPE estado_pedido AS ENUM ('pendiente', 'enviado', 'cancelado');
```

- Ejemplo:

```
CREATE TABLE pedidos (  
    estado estado_pedido  
);
```

- La columna estado_pedido solo podrá almacenar los valores 'pendiente', 'enviado' o 'cancelado'.

8. TIPOS DE DATOS



- Elementos de las sentencias SQL
 - Casi todas las sentencias SQL tienen una forma básica:
 - Empiezan por un verbo (palabra reservada que describe lo que hace la sentencia)
 - (ejemplo SELECT)
 - A continuación le siguen una o más cláusulas que explican más detalles acerca de la sentencia
 - También empiezan con una palabra clave, (por ejemplo FROM o WHERE).
 - Algunas de estas cláusulas son opcionales y otras obligatorias

8. TIPOS DE DATOS



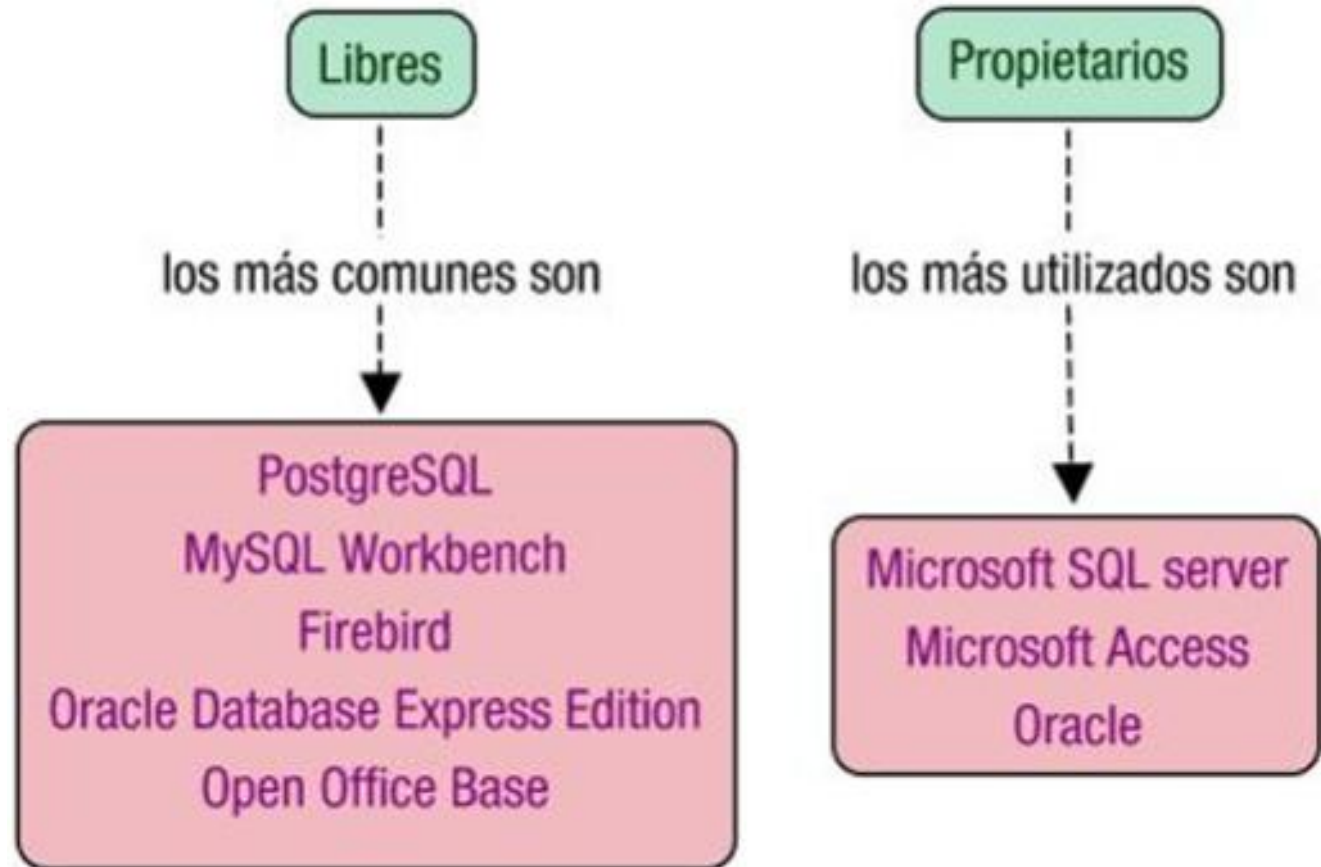
- Elementos de las sentencias SQL
 - Casi todas las sentencias SQL tienen una forma básica:
 - Las fechas se representan entre comillas y en formato numérico usando como separador el guión.
 - Por ejemplo, el 17 de enero de 2011 se representa: "2011-1-17"
 - Las horas (datos de tipos TIME) se representan entre comillas y usando como separador el carácter de dos puntos.
 - Por ejemplo, para representar las 4 y 20 de la tarde: "16:20:00"

ACTIVIDAD 2

- Tipos de datos en tablas MySQL

9. HERRAMIENTAS GRÁFICAS PARA EL DISEÑO FÍSICO

- Podemos establecer dos grupos:



9. HERRAMIENTAS GRÁFICAS PARA EL DISEÑO FÍSICO

- **Previo al modelo físico, se debe usar una herramienta gráfica para modelar, crear y administrar bases de datos.**
 - Primero diseñar el diagrama ER
 - Obtener el diagrama del esquema relacional
 - Y también las sentencias de creación de tablas, vistas e índices de manera automática.

9. HERRAMIENTAS GRÁFICAS PARA EL DISEÑO FÍSICO

- pgAdmin4
 - Actualmente puede:
 - Crear y eliminar Bases de Datos
 - Crear, eliminar y modificar tablas
 - Borrar, editar y añadir campos
 - Ejecutar cualquier sentencia SQL
 - Administrar claves en campos
 - Administrar privilegios
 - Exportar datos en varios formatos y está disponible en 62 idiomas

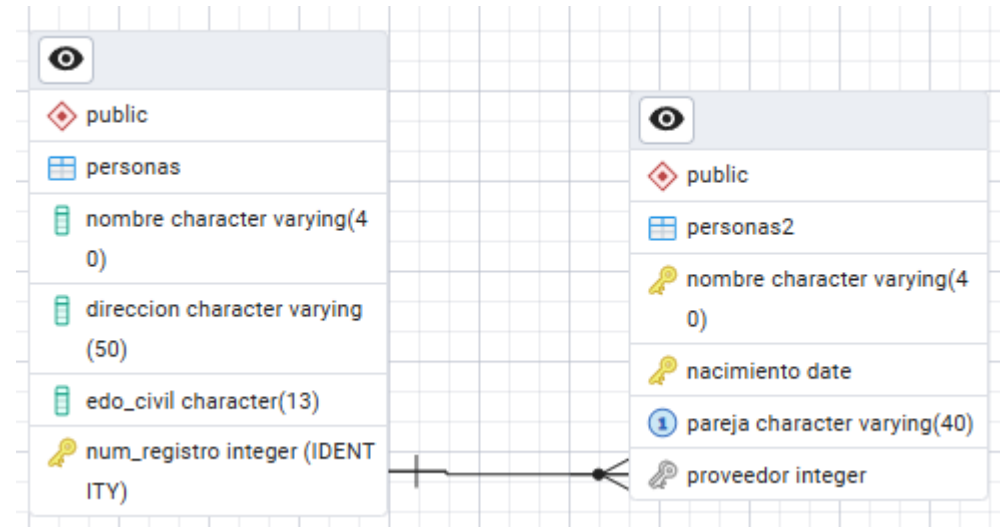
9. HERRAMIENTAS GRÁFICAS PARA EL DISEÑO FÍSICO

- **Actividad:**

- Crear dos tablas
 - Relacionar las dos tablas
 - Crear un script
 - Revisar y comprender todas las órdenes del script
 - Crear el diccionario de datos
-

9. HERRAMIENTAS GRÁFICAS PARA EL DISEÑO FÍSICO

- pgAdmin4
 - Diagrama ER realizado:
 - Llave amarilla: PK
 - Llave gris: FK



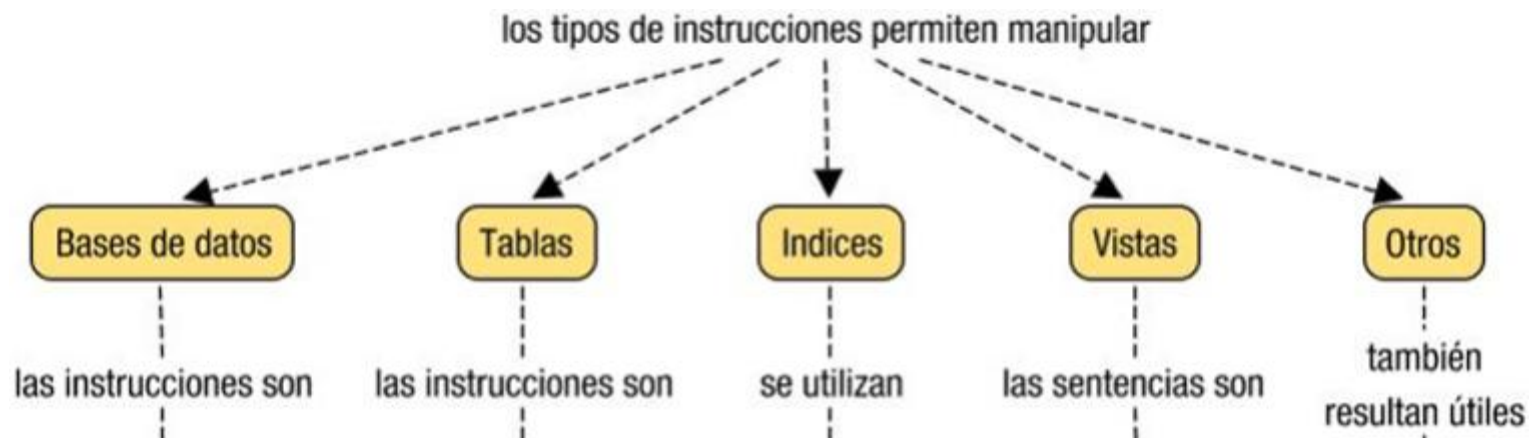
ACTIVIDAD 3

- Herramientas gráficas



10. ÍNDICES

- DDL - Lenguaje de Definición de Datos



10. ÍNDICES

- **Un índice permite consultar las filas de una tabla de una forma más rápida.**
 - **Almacenamiento:** La información sobre los índices se almacena en una **tabla de índices** de tamaño mucho más reducido que la tabla de datos, ordenados de forma ascendente junto con información relativa a donde están almacenados los datos correspondientes a cada valor del índice.
 - **Acceso:** El usuario no puede acceder a la tabla de índices, aunque siempre que realiza una operación condicionada a los valores de un índice, el servidor **PostgreSQL busca primero en la tabla de índices para, a través de la información almacenada en el índice, acceder de forma directa a los datos correspondientes de la tabla de datos.**
-

10. ÍNDICES

- Ventajas y Desventajas
 - Si bien un índice **permite consultar de forma más rápida la tabla**, las operaciones de añadir filas, modificar filas o eliminar filas consumen más tiempo.
 - Además, los datos de una tabla en la que se definen índices ocupan **más espacio** que el dedicado exclusivamente a esos datos, puesto que necesita un espacio adicional para almacenar una tabla de índices.
 - Por ello, no es aconsejable definir índices de manera indiscriminada. Lo que se debe hacer es **definir índices sobre columnas a partir de las cuales se vayan a hacer consultas frecuentes**.
-

10. ÍNDICES

- Los índices pueden crearse:
 - Al crear la tabla
 - Al modificar la estructura de la tabla
 - Creándose directamente mediante la sentencia `CREATE INDEX`
-

10. ÍNDICES

- Cuando se generan claves primarias o foráneas se crean automáticamente índices.
- El nombre del índice debe ser **único** en la base de datos, sino dará error.

10. ÍNDICES

- PRIMARY KEY
 - Una tabla sólo puede tener un índice PK
 - PostgreSQL crea automáticamente un índice para la clave primaria, con un nombre interno generado por el sistema.
 - En una sentencia CREATE TABLE, se **establece primero el índice asociado a la PK**, luego los índices UNIQUES y posteriormente los NON UNIQUE.
 - Esto ayuda al optimizador de PostgreSQL a priorizar qué índice usar y también más rápidamente para detectar claves ÚNICAS duplicadas.
-

10. ÍNDICES

- PRIMARY KEY
 - Una PK puede ser un índice de **varias columnas**
 - Sin embargo, no puede crear un índice de varias columnas utilizando el atributo de clave PRIMARY KEY en una especificación de columna. Hacerlo solo marca esa única columna como principal.
 - Debe usar una cláusula PRIMARY KEY(key_part, ...) **separada**.

10. ÍNDICES

- CREATE INDEX
 - Normalmente, se crean todos los índices en una tabla cuando se crea la tabla con la sentencia: CREATE TABLE.
 - **CREATE INDEX le permite agregar índices a las tablas existentes.**

10. ÍNDICES

- CREATE INDEX
 - CREATE INDEX no se puede utilizar para crear una PRIMARY KEY
 - Para eso se utiliza: ALTER TABLE
 - ALTER TABLE Persons ADD PRIMARY KEY (ID);

10. ÍNDICES

- CREATE INDEX

- **Permite añadir índices a tablas existentes.**
- Un índice, al ser creado, puede recibir un nombre. Si no se especifica ese nombre, el índice recibe el nombre de la primera columna que forma ese índice.
- Un índice se puede definir sobre una columna o sobre un grupo de columnas.
- Para los índices formados por columnas de tipo CHAR o VARCHAR pueden crearse índices por expresión.

```
CREATE INDEX idx_apellidos_5 ON personas ((left(apellidos, 5)));
```

10. ÍNDICES

- CREATE INDEX

```
1 CREATE [UNIQUE | FULLTEXT | SPATIAL] INDEX index_name
2     [index_type]
3     ON tbl_name (key_part, ...)
4     [index_option]
5     [algorithm_option | lock_option] ...
6
7 key_part: {col_name [(length)] | (expr)} [ASC | DESC]
8
```

- Una especificación `key_part` puede terminar con `ASC` o `DESC` para especificar si los valores de índice se almacenan en orden ascendente o orden descendiente.
 - El valor **predeterminado es ascendente** si no se proporciona ningún especificador de orden.
-

10. ÍNDICES

- DROP INDEX

- Elimina un índice de una tabla.

```
DROP INDEX [CONCURRENTLY] [IF EXISTS] index_name [CASCADE|RESTRICT];
```

- DROP INDEX elimina el índice llamado index_name.
-

10. ÍNDICES

- DROP INDEX
 - EJEMPLO
 - CREATE INDEX idx_apellidos ON Personas (apellidos);
 - Podría poner más campos que formen el índice.
 - DROP INDEX idx_apellidos;

11. VISTAS

- Una vista es una tabla virtual definida mediante una consulta SQL.
 - PostgreSQL permite crear, modificar y eliminar vistas mediante las sentencias CREATE VIEW, CREATE OR REPLACE VIEW y DROP VIEW
 - Consiste en crear tablas virtuales a partir de consultas SELECT a otras tablas.
 - Las vistas de una base de datos PostgreSQL tienen la misma estructura de filas y columnas de una tabla – **no puede haber una vista con el mismo nombre de una tabla.**
-

11. VISTAS

```
CREATE VIEW test.v AS SELECT * FROM t;
```

- Las vistas se usan para poder evitar que determinados usuarios accedan a ciertas tablas, dándoles acceso solamente a una parte de esas tablas a través de las vistas.

- EJEMPLO

```
1  mysql> CREATE TABLE t (qty INT, price INT);
2  mysql> INSERT INTO t VALUES(3, 50);
3  mysql> CREATE VIEW v AS SELECT qty, price, qty*price AS value FROM t;
4  mysql> SELECT * FROM v;
5  +-----+-----+-----+
6  | qty | price | value |
7  +-----+-----+-----+
8  |   3 |    50 |   150 |
9  +-----+-----+-----+
```

- El siguiente ejemplo define una vista que selecciona dos columnas de otra tabla, así como una expresión calculada a partir de esas columnas.
-

11. VISTAS

- DROP VIEW
- DROP VIEW elimina una o más vistas:

```
1 DROP VIEW [IF EXISTS]
2   view_name [, view_name] ...
3   [RESTRICT | CASCADE]
```

12. DICCIONARIO DE DATOS

- Una vez realizado el diseño físico de la base de datos, antes de dar por concluido el trabajo de diseño e implantación, se debe confeccionar el **diccionario de datos**.
 - En él se hace una **descripción detallada de todos los elementos que intervienen en el sistema** y nos servirá para tener una visión global de la base de datos, sin ambigüedad.
 - Esta información puede ser útil tanto para el diseñador, como para otras personas que se incorporen al proyecto.
-

12. DICCIONARIO DE DATOS

- En el diccionario de datos se describen los atributos y, para cada uno de ellos, se tiene:
 - Su dominio: tipo de datos, longitud y restricciones de dominio.
 - El valor por defecto, que es opcional.
 - Si admite nulos.
 - Si es derivado y, en caso de serlo, cómo se calcula su valor

12. DICCIONARIO DE DATOS

- INFORMATION_SCHEMA
 - En PostgreSQL la información relativa a los metadatos de la base de datos se encuentra en **INFORMATION_SCHEMA**.
 - **Se trata de un esquema que contiene vistas estándar con metadatos de los objetos de la base de datos actual.**
 - Cada usuario tiene derecho a acceder a estas tablas, pero sólo a los registros que se refieren a los objetos a los que tiene permiso de acceso.
 - Se puede acceder a su contenido con SELECT pero no se puede insertar, actualizar o borrar.
-

12. DICCIONARIO DE DATOS

- INFORMATION_SCHEMA

- Listar tablas del esquema public:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public';
```

- Ver la estructura de una tabla:

```
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'public'
AND table_name = 'tabla';
```
